

Trabajo Práctico 1

Procesamiento de Imágenes y Visión por Computadora

Matías Cisnero

11 de agosto de 2025

Consigna 1:

Implementar el detector de bordes por el método del gradiente utilizando los siguientes operadores de gradiente:

- 1 Prewitt.
- 2 Sobel.



Los operadores de gradiente se basan en calcular estimaciones del **gradiente**.

En imágenes se calcula como:

$$I : \mathbb{R}^2 \rightarrow [0, \dots, 255]$$

$$\nabla I = (I_x, I_y)$$

El vector ∇I representa el gradiente de la imagen, y para cada píxel apunta en la dirección de mayor cambio de intensidad.

I_x y I_y se pueden aproximar con los operadores de **Prewitt** y de **Sobel**.

Magnitud del gradiente

Para cada píxel se calcula:

$$|\nabla I| = \sqrt{I_x^2 + I_y^2}$$

Este valor indica la magnitud del cambio en la intensidad.



Estos corresponden a las diferencias en secciones de 3×3

Operadores de Prewitt

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Los operadores de Sobel le dan más importancia a los píxeles más cercanos al centro.

Operadores de Sobel

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$



El código detectar los bordes de una imagen mediante los operadores de gradiente es:

```
def aplicar_magnitud_del_gradiente(imagen_np: np.ndarray,
    func_filtro1, func_filtro2) -> np.ndarray:

    ix = aplicar_filtro(imagen_np, func_filtro=func_filtro1,
        modo=2)
    iy= aplicar_filtro(imagen_np, func_filtro=func_filtro2,
        modo=2)

    magnitud = np.sqrt((ix**2)+(iy**2))

    resultado_np = escalar_255(magnitud)

    return resultado_np
```



Al aplicar los **Operadores de gradiente Prewitt y Sobel** sobre la imagen de la Figura 1, se obtuvieron las versiones modificadas visibles en la Figura 2 y Figura 3.



Figura 1: Original



Figura 2: Prewitt



Figura 3: Sobel

(*) No se observan cambios significativos entre ambos operadores de gradiente.

Consigna 2:

Aplicar los detectores de borde del punto anterior a las mismas imágenes contaminadas con ruido.

Al aplicar los **Operadores de gradiente Prewitt y Sobel** sobre la imagen con ruido Gaussiano (40 % de contaminación y $\sigma = 30$) de la Figura 4, se obtuvieron las versiones modificadas visibles en la Figura 5 y Figura 6.

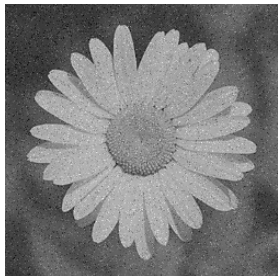


Figura 4: Ruido

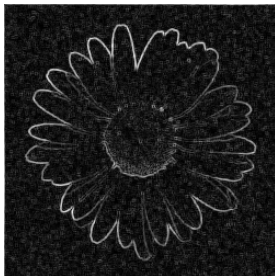


Figura 5: Prewitt

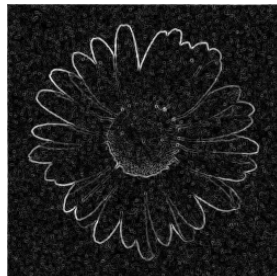


Figura 6: Sobel

(*) No se observan cambios significativos entre ambos operadores de gradiente.

Consigna 3:

Aplicar los detectores de borde del punto anterior a imágenes en color.



Al aplicar los **Operadores de gradiente Prewitt y Sobel** sobre la imagen a color de la Figura 7, se obtuvieron las versiones modificadas visibles en la Figura 8 y Figura 9.



Figura 7: Ruido



Figura 8: Prewitt



Figura 9: Sobel

(*) No se observan cambios significativos entre ambos operadores de gradiente.

Consigna 4:

Implementar los siguientes detectores de borde y aplicarlos a dos imágenes y a sus versiones contaminadas:

- 1 Método del Laplaciano.
- 2 Método del Laplaciano agregándole evaluación de la pendiente.
- 3 Método del Laplaciano del Gaussiano (Marr-Hildreth).



Operador de Laplace

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

$$\Delta f = \frac{\partial^2 f}{\partial^2 x} + \frac{\partial^2 f}{\partial^2 y}$$

En imágenes se aproxima como:

$$\Delta I(x, y) = 4 * I(x, y) - I(x - 1, y) - I(x + 1, y) - I(x, y - 1) - I(x, y + 1)$$

(*) El operador Laplaciano es la divergencia del gradiente.



Como una máscara esto es:

Máscara de Laplace

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Luego de convolucionar la imagen con la máscara de Laplace se deben encontrar los cruces por cero, el proceso es:

- Recorrer la imagen por filas y detectar los cambios de signos entre dos valores consecutivos, si hay un cambio asignar a ese pixel el valor 255, sino asignar el valor 0.
- Repetir para todas las filas.

Al encontrar los cruces por cero la imagen resulta en una binaria.



```
def encontrar_cruces_por_cero(imagen_np: np.ndarray) -> np
    .ndarray:
    m, n, _ = imagen_np.shape
    imagen_filtrada = np.zeros_like(imagen_np) #
        Predeterminadamente son cero y solo cambio los 255

    for i in range(m):
        for j in range(n - 1):
            for c in range(3):
                if imagen_np[i, j, c] * imagen_np[i, j + 1, c] <
                    0:
                        imagen_filtrada[i, j] = 255

    return imagen_filtrada # Retorna una img binaria
```



Al encontrar los cruces por cero se producen muchos puntos de borde falsos. Para corregir esto se mide el valor absoluto de la **pendiente del cruce** y si este valor supera cierto umbral se lo considera un borde.

Pendiente del cruce

$$|a + b|$$



```
def encontrar_cruces_por_cero_pendiente(imagen_np: np.
    ndarray, umbral=128) -> np.ndarray:
    m, n, _ = imagen_np.shape
    imagen_filtrada = np.zeros_like(imagen_np) #
        Predeterminadamente son cero y solo cambio los 255

    for i in range(m):
        for j in range(n - 1):
            for c in range(3):
                if abs(imagen_np[i, j, c]) + abs(imagen_np[i, j +
                    1, c]) > umbral:
                    imagen_filtrada[i, j] = 255

    return imagen_filtrada # Retorna una img binaria
```



Para aplicar el Laplaciano del Gaussiano a una imagen debemos construir la máscara del LoG.

Calculo del Laplaciano del Gaussiano

$$\Delta G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^3} e^{-\frac{x^2+y^2}{2\sigma^2}} \left(\frac{x^2+y^2}{\sigma^2} - 2 \right)$$

Luego se aplica la máscara a la imagen y se hacen los cruces por cero.

Sobre la imagen de la Figura 10 se aplicaron los siguientes detectores de borde:

- Figura 11: Método del Laplaciano.
- Figura 12: Método del Laplaciano con evaluación de la pendiente con umbral=128.
- Figura 13: Método del Laplaciano del Gausiano (Marr-Hildreth) con $\sigma = 1$.

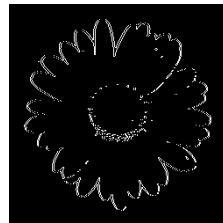
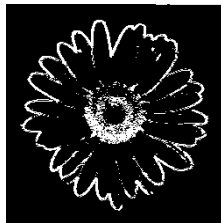
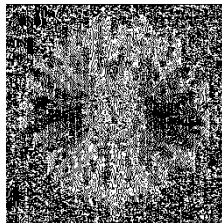


Figura 10: Original

Figura 11: Lap.

Figura 12: Lap. P.

Figura 13: LoG



Sobre la imagen de la Figura 14 la cual presenta ruido Gaussiano (40 % de contaminación y $\sigma = 30$), se aplicaron los siguientes detectores de borde:

- Figura 15: Método del Laplaciano.
- Figura 16: Método del Laplaciano con evaluación de la pendiente con umbral=128.
- Figura 17: Método del Laplaciano del Gausiano (Marr-Hildreth) con $\sigma = 1$.

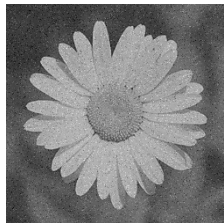


Figura 14: Ruido

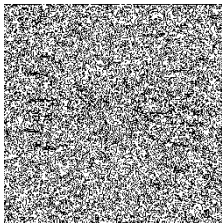


Figura 15: Lap.



Figura 16: Lap. P.

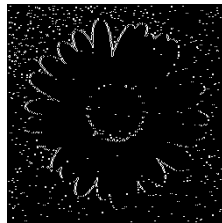


Figura 17: LoG

Sobre la imagen de la Figura 18 se aplicaron los siguientes detectores de borde:

- Figura 19: Método del Laplaciano.
- Figura 20: Método del Laplaciano con evaluación de la pendiente con umbral=40.
- Figura 21: Método del Laplaciano del Gausiano (Marr-Hildreth) con $\sigma = 1$.

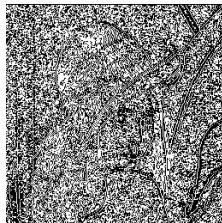
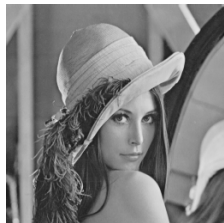


Figura 18: Original

Figura 19: Lap.

Figura 20: Lap. P.

Figura 21: LoG



Sobre la imagen de la Figura 22 la cual presenta ruido Gaussiano (40 % de contaminación y $\sigma = 20$), se aplicaron los siguientes detectores de borde:

- Figura 23: Método del Laplaciano.
- Figura 24: Método del Laplaciano con evaluación de la pendiente con umbral=40.
- Figura 25: Método del Laplaciano del Gausiano (Marr-Hildreth) con $\sigma = 1$.



Figura 22: Ruido

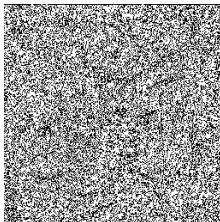


Figura 23: Lap.

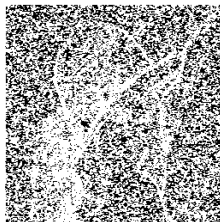


Figura 24: Lap. P.



Figura 25: LoG

Consigna 5:

Implementar los métodos de Difusión Isotrópica y Anisotrópica. Aplicarlos a imágenes con ruido gaussiano y con ruido sal y pimienta. Comparar con el filtro de la mediana.



La difusión isotrópica resuelve la ecuación del calor de conducción isotrópica:

Difusión Isotrópica

$$I_t(x, y, t) = \Delta I(x, y) = I_{xx} + I_{yy}$$

Con la condición inicial: $I(x, y, 0) = I_0$

Por otra parte la difusión anisotrópica

Difusión Anisotrópica

$$I_t(x, y, t) = \nabla \cdot (c(x, y, t) \nabla I(x, y))$$

Con la condición inicial: $I(x, y, 0) = I_0$

$c(x, y, t)$ es el coeficiente de conducción, el cual tiene como objetivo tener comportamiento distinto dependiendo si está en un borde o no (0 o 1 respectivamente).

(*) Difusión del calor, con coeficiente constante o variable.



Para estimar la presencia de los bordes utilizaremos el modulo del gradiente:

Coeficiente

$$c(x, y, t) = g(|\nabla I|)$$

Y como detectores del mismo tenemos dos opciones:

Detector de Leclerc:

$$g(|\nabla I|) = e^{\frac{-|\nabla I|^2}{\sigma^2}}$$

Detector de Lorentz:

$$g(|\nabla I|) = \frac{1}{\frac{-|\nabla I|^2}{\sigma^2} + 1}$$

(*) .



La implementación consiste en calcular:

Implementación de Perona - Malik

$$I^{t+1}(x, y) = I^t(x, y) + \lambda(D_N c_N + D_E c_E + D_O c_O + D_S c_S)$$

T veces.

Donde

$$D_N = I(x, y + 1) - I(x, y) \quad D_E = I(x - 1, y) - I(x, y)$$

$$D_O = I(x + 1, y) - I(x, y) \quad D_S = I(x, y - 1) - I(x, y)$$

Y los coeficientes se calculan como

$$c_N = g(D_N) \quad c_E = g(D_E) \quad c_O = g(D_O) \quad c_S = g(D_S)$$

Si el filtro es isotrópico $c_N = c_E = c_O = c_S = 1$

(*) $\lambda = 0,25$



```
def detector_de_leclerc(gradiente, sigma:int):  
    return np.exp(-(gradiente**2)/sigma**2)  
  
def detector_de_lorentz(gradiente, sigma:int):  
    return 1/(((-(gradiente**2)/sigma**2) + 1)
```



Se ejecuta t veces el siguiente código para cada pixel (i, j, c) de la imagen:

```
# Gradientes
DN = img_filt[i, j + 1, c] - img_filt[i, j, c]
DE = img_filt[i - 1, j, c] - img_filt[i, j, c]
DO = img_filt[i + 1, j, c] - img_filt[i, j, c]
DS = img_filt[i, j - 1, c] - img_filt[i, j, c]

# Coeficientes
if isotropico: cN = cE = cO = cS = 1
else:
    cN = detector_de_leclerc(DN, sigma)
    cE = detector_de_leclerc(DE, sigma)
    cO = detector_de_leclerc(DO, sigma)
    cS = detector_de_leclerc(DS, sigma)

# Actualización
img_filt[i, j, c] += lamb * (DN * cN + DE * cE + DO * cO +
    DS * cS)
```



Sobre la imagen de la Figura 26 la cual presenta ruido Gaussiano (30 % de contaminación y $\sigma = 20$), se aplicaron los siguientes métodos de eliminación de ruido:

- Figura 27: Método de Difusión Isotrópica con $t = 2$.
- Figura 28: Método de Difusión Anisotrópica con $t = 5$ y $\sigma = 40$.
- Figura 29: Filtro de la Mediana con $k = 3$.



Figura 26: Ruido



Figura 27: Iso.



Figura 28: Anis.

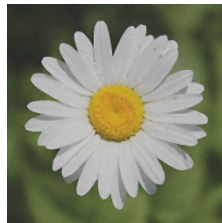


Figura 29: Mediana

Sobre la imagen de la Figura 30 la cual presenta ruido Sal y Pimienta ($p = 10$), se aplicaron los siguientes métodos de eliminación de ruido:

- Figura 31: Método de Difusión Isotrópica con $t = 15$.
- Figura 32: Método de Difusión Anisotrópica con $t = 10$ y $\sigma = 50$.
- Figura 33: Filtro de la Mediana con $k = 5$.

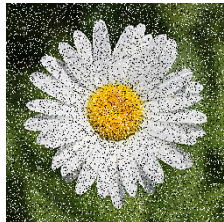


Figura 30: Ruido



Figura 31: Iso.

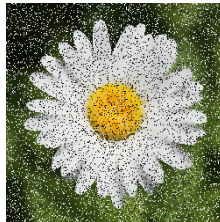


Figura 32: Anis.

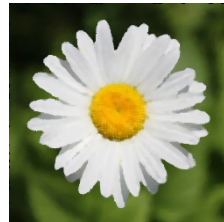


Figura 33: Mediana

Consigna 6:

Implementar el filtro bilateral. Aplicarlo a imágenes con ruido gaussiano y con ruido sal y pimienta. Comparar con el filtro de difusión anisotrópica.



Es un filtro donde el valor de cada pixel se reemplaza por el promedio pesado de los pixeles vecinos, el peso depende de la distancia de los pixeles al centro y de la diferencia en la intensidad del color.

Filtro Bilateral

$$I^{filt}(x) = \frac{1}{W_x} \sum_{x_i \in \Omega} I(x_i) G_{\sigma_s}(\|x_i - x\|) G_{\sigma_r}(\|I(x_i) - I(x)\|)$$

Donde

$$W_x = \sum_{x_i \in \Omega} G_{\sigma_s}(\|x_i - x\|) G_{\sigma_r}(\|I(x_i) - I(x)\|)$$

W_x es un factor de normalización.



Luego los filtros se construyen como:

Filtros

$$G_{\sigma_s} = e^{-\frac{(x^1 - x_i^1)^2 + (x^2 - x_i^2)^2}{2\sigma_s^2}}$$

$$G_{\sigma_r} = e^{-\frac{\|I(x) - I(x_i)\|^2}{2\sigma_r^2}}$$

(*) Recordar que $I(x)$ es un vector.



Para cada pixel (i, j) se calcula su valor como:

```
region = imagen_padded[i:i+k, j:j+1, :]  
  
dif = region - region[pad_h, pad_w, :]  
  
Gr = np.exp(-(np.sum(dif**2, axis=2) / (2 * sigma_r**2)))  
  
Wx = np.sum(Gs * Gr)  
  
G = Gs * Gr  
G = G[:, :, np.newaxis]  
  
valor = (1 / Wx) * np.sum(region * G, axis=(0, 1))  
  
imagen_filtrada[i, j, :] = valor
```

(*) Gs está calculado de ante mano por eficiencia.



Sobre la imagen de la Figura 34 la cual presenta ruido Gaussiano (30 % de contaminación y $\sigma = 20$), se aplicaron los siguientes métodos de eliminación de ruido:

- Figura 35: Filtro bilateral con $\sigma_s = 10$ y $\sigma_r = 50$.
- Figura 36: Método de Difusión Anisotrópica con $t = 5$ y $\sigma = 40$.



Figura 34: Ruido



Figura 35: Bilateral



Figura 36: Anis.

Se observan cambios significativos entre ambos métodos, particularmente el Filtro Bilateral conserva mejor los bordes y la textura.

Sobre la imagen de la Figura 37 la cual presenta ruido Sal y Pimienta ($p = 10$), se aplicaron los siguientes métodos de eliminación de ruido:

- Figura 38: Filtro bilateral con $\sigma_s = 10$ y $\sigma_r = 50$.
- Figura 39: Método de Difusión Anisotrópica con $t = 10$ y $\sigma = 50$.

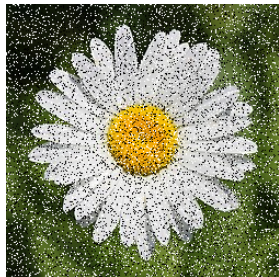


Figura 37: Ruido



Figura 38: Bilateral

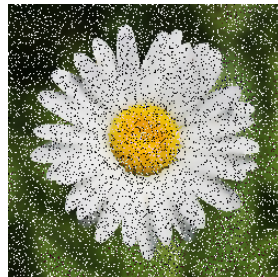


Figura 39: Anis.

Se observa que ninguno de estos métodos puede eliminar el ruido Sal y Pimienta.

Consigna 7:

Implementar los siguientes algoritmos de umbralización y aplicarlos a dos imágenes y a sus versiones contaminadas:

- 1 Umbralización óptima iterativa.
- 2 Método de umbralización de Otsu.
- 3 Segmentación de imágenes en color (RGB) utilizando umbralización por bandas.



```
for i in range(n):  
    if T == T_anterior: break  
    T_anterior = T  
  
    imagen_binaria = aplicar_umbralizacion(imagen_np, T)  
    nG1 = np.sum(imagen_binaria == 255)  
    nG2 = np.sum(imagen_binaria == 0)  
  
    m1 = (1 / nG1) * np.sum(imagen_np[imagen_binaria ==  
        255])  
    m2 = (1 / nG2) * np.sum(imagen_np[imagen_binaria == 0])  
  
    T = int(0.5 * (m1 + m2))
```

(*) A manera de establecer un tope $n=50$.



```
intensidad = np.arange(256)
fi = np.bincount(datos_gris, minlength=256)
N = datos_gris.size
pi = fi / N

P1 = np.zeros(256)
m = np.zeros(256)
for t in range(256):
    P1[t] = np.sum(pi[0:t+1])
    m[t] = np.sum(intensidad[0:t+1] * pi[0:t+1])
mG = m[255] # np.sum(intensidad * pi)

sigma_B = np.zeros(256)
for t in range(256):
    if P1[t] > 0 and P1[t] < 1:
        sigma_B[t] = ((mG * P1[t] - m[t])**2) / (P1[t] * (1 -
            P1[t]))
t_estrella = np.argmax(sigma_B)
resultado_np = aplicar_umbralizacion(imagen_np, t_estrella
)
```



```
def aplicar_umbralizacion_rgb(imagen_np: np.ndarray) -> np
    .ndarray:
    banda_r = imagen_np[:, :, 0]
    banda_g = imagen_np[:, :, 1]
    banda_b = imagen_np[:, :, 2]

    banda_r = aplicar_umbralizacion_de_otsu(banda_r)
    banda_g = aplicar_umbralizacion_de_otsu(banda_g)
    banda_b = aplicar_umbralizacion_de_otsu(banda_b)

    resultado_np = np.dstack([banda_r, banda_g, banda_b])

    # https://numpy.org/doc/stable/reference/generated/numpy
    .dstack.html
    return resultado_np
```



Sobre la imagen de la Figura 40 se aplicaron los siguientes métodos de umbralización:

- Figura 41: Umbralización óptima iterativa, obteniendo $T = 136$ en la iteración 3.
- Figura 42: Método umbralización de Otsu, obteniendo $T = 136$.
- Figura 43: Segmentación de imágenes en color (RGB) utilizando umbralización por bandas, obteniendo estos umbrales para cada canal: R=136, G=142 y B=117.

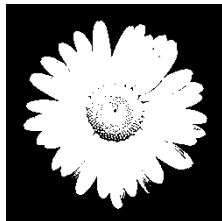


Figura 40: Original

Figura 41: Iter.

Figura 42: Otsu.

Figura 43: Seg.



Sobre la imagen de la Figura 44 contaminada, se aplicaron los siguientes métodos de umbralización:

- Figura 45: Umbralización óptima iterativa, obteniendo $T = 128$ en la iteración 3.
- Figura 46: Método umbralización de Otsu, obteniendo $T = 128$.
- Figura 47: Segmentación de imágenes en color (RGB) utilizando umbralización por bandas, obteniendo estos umbrales para cada canal: R=128, G=132 y B=116.



Figura 44: Ruido



Figura 45: Iter.



Figura 46: Otsu.



Figura 47: Seg.



Sobre la imagen de la Figura 48 se aplicaron los siguientes métodos de umbralización:

- Figura 49: Umbralización óptima iterativa, obteniendo $T = 123$ en la iteración 4.
- Figura 50: Método umbralización de Otsu, obteniendo $T = 125$.
- Figura 51: Segmentación de imágenes en color (RGB) utilizando umbralización por bandas, obteniendo estos umbrales para cada canal: R=119, G=126 y B=138.



Figura 48: Original



Figura 49: Iter.



Figura 50: Otsu.

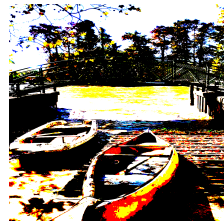


Figura 51: Seg.



Sobre la imagen de la Figura 52 se aplicaron los siguientes métodos de umbralización:

- Figura 53: Umbralización óptima iterativa, obteniendo $T = 113$ en la iteración 2.
- Figura 54: Método umbralización de Otsu, obteniendo $T = 115$.
- Figura 55: Segmentación de imágenes en color (RGB) utilizando umbralización por bandas, obteniendo estos umbrales para cada canal: R=112, G=115 y B=122.



Figura 52: Ruido



Figura 53: Iter.



Figura 54: Otsu.

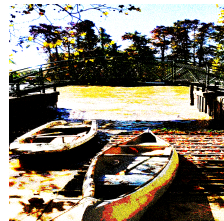


Figura 55: Seg.



Todo el código y los resultados de este trabajo están disponibles en mi repositorio de GitHub:

https://github.com/matias-cisnero/procesamiento_imagenes



NumPy Documentation: Broadcasting.

<https://numpy.org/doc/stable/user/basics.broadcasting.html>



Imagen "*Leucanthemum vulgare* Blüte" por Mariofan13.

Wikimedia Commons. Licencia *CC BY-SA 3.0 Unported*.

[Ver imagen en Wikimedia Commons](#)



Gonzalez, R. and Woods, R. (2018). Digital Image Processing. Pearson Education 4th Edition, New York.

¡Gracias!

